Практическая работа 3

ASP.NET Core (.NET 10)

Тема: middleware, ветвление pipeline, окружение и dependency injection

Сквозное задание: "Учебный диагностический веб-сервис StudentPortal.Diagnostics"

Работа собрана по мотивам тем: Use, UseWhen, MapWhen, Map, классы middleware, порядок pipeline, IWebHostEnvironment, IServiceCollection и создание сервисов.

Паспорт работы
Формат: одна интегрированная практическая работа вместо набора отдельных мелких упражнений.
Цель: научиться проектировать и собирать конвейер обработки запроса, выносить middleware в классы, ветвить pipeline по пути и условию, использовать окружение приложения и регистрировать собственные сервисы.
Средства: .NET 10 SDK, шаблон Empty Web App (dotnet new web), Visual Studio / Rider / VS Code, браузер или Postman / Swagger.
Результат: рабочее веб-приложение и краткий отчет с объяснением порядка middleware и тестовых запросов.

1. Что должен освоить студент

- понимать разницу между терминальным и нетерминальным middleware;
- реализовывать inline middleware через app.Use(...) и управлять вызовом next;
- ветвить конвейер по условию через UseWhen(...) и MapWhen(...);
- ветвить конвейер по пути через Map(...) и создавать вложенные ветки;
- выносить middleware в отдельные классы и подключать их через методы расширения;
- объяснять, почему порядок middleware влияет на итог обработки запроса;
- получать информацию об окружении приложения через IWebHostEnvironment;
- регистрировать собственные сервисы в IServiceCollection и использовать их в приложении.

2. Сквозной сценарий задания

Необходимо создать веб-приложение StudentPortal.Diagnostics. Это учебный сервис диагностики внутреннего портала кафедры. Приложение должно уметь отдавать время и дату, показывать информацию об окружении, выводить сведения о зарегистрированных сервисах, обрабатывать защищенные запросы по токену и демонстрировать разные способы ветвления pipeline.

3. Какие темы и где именно отрабатываются

Тема	Что реализует студент	Как проверить
Use()	Inline middleware, который замеряет время обработки запроса и записывает отметку до и после next.	В консоли или ответе видно, что код выполняется до и после следующего middleware.
UseWhen()	Ветка для запросов с trace=true: логирование и	Запрос /tools/time?trace=true

	добавление заголовка X-Debug-Trace, затем возврат в основной pipeline.	обрабатывается веткой и затем обычным маршрутом.
MapWhen()	Отдельная ветка для format=plain, которая отдает специальный текстовый ответ и не продолжает основной pipeline.	Любой запрос с format=plain отрабатывает в отдельной ветке.
Map()	Ветка /tools и вложенные ветки /tools/time, /tools/date, /tools/info; отдельная ветка /secure.	Пути /tools/* и /secure/* работают независимо от основной ветки.
Классы middleware	TokenMiddleware и ErrorHandlingMiddleware как отдельные классы.	При неверном токене или неизвестном пути возвращаются ожидаемые статусы и сообщения.
Порядок pipeline	ErrorHandlingMiddleware должен стоять раньше остальных; токен-проверка должна выполняться раньше защищенного обработчика.	Студент в отчете объясняет, что изменится при перестановке компонентов.
IWebHostEnvironment	Endpoint /env показывает EnvironmentName, ApplicationName, ContentRootPath, WebRootPath.	При смене среды отображаются другие значения или меняется поведение ответа.
IServiceCollection и сервисы	Регистрация собственных сервисов и endpoint /di/services.	В выводе видно наличие собственных и встроенных сервисов.

4. Функциональные требования к приложению

1. Создать проект шаблона Empty Web App под .NET 10.
2. Настроить inline middleware через app.Use(...), который делает две вещи: а) фиксирует момент начала обработки; б) после next добавляет запись о завершении обработки.
3. Создать ветку UseWhen(...), которая срабатывает только если в query string есть trace=true. В этой ветке необходимо выполнить дополнительное логирование запроса и затем вернуть выполнение в основной pipeline.
4. Создать ветку MapWhen(...), которая срабатывает при наличии format=plain. Ветка должна формировать самостоятельный текстовый ответ и не использовать основной маршрут.
5. Создать ветку Map("/tools", ...), внутри которой должны быть вложенные маршруты /tools/time, /tools/date и /tools/info.
6. Создать ветку Map("/secure", ...), в которой подключается custom middleware TokenMiddleware через собственный метод расширения UseToken(...).
7. Реализовать ErrorHandlingMiddleware, который после выполнения следующих компонентов проверяет StatusCode и дописывает понятное сообщение для 403 и 404.
8. Создать endpoint /env с выводом данных из IWebHostEnvironment.
9. Создать endpoint /di/services, который показывает хотя бы общее число сервисов и не менее 10 записей из builder.Services.
10. Зарегистрировать минимум два собственных сервиса и вынести их регистрацию в метод расширения AddStudentPortalServices(...).

5. Обязательные маршруты и ожидаемое поведение

Запрос	Ожидаемое поведение	Какая тема проверяется
/	Стартовая страница приложения. Краткое описание проекта и ссылки на тестовые маршруты.	Терминальный обработчик
/tools/time	Возвращает текущее время через сервис.	Map + DI
/tools/date	Возвращает текущую дату через сервис.	Map + DI
/tools/info	Отдает краткую диагностическую информацию о приложении.	Вложенные ветки Map
/tools/time?trace=true	Сначала выполняется ветка UseWhen, затем основной обработчик /tools/time.	UseWhen
/anything?format=plain	Запрос должен быть перехвачен отдельной веткой и вернуть plain-text ответ.	MapWhen
/secure/report	Без токена должен вернуть 403 и текст ошибки.	Класс middleware
/secure/report?token=study2026	При валидном токене доступ разрешен.	Класс middleware + extension
/env	Показывает сведения об окружении и путях приложения.	IWebHostEnvironment
/di/services	Выводит сведения о сервисах из IServiceCollection.	IServiceCollection
/unknown	Возвращает 404 и сообщение Not Found.	Порядок pipeline + ErrorHandlingMiddleware

6. Минимальный набор собственных классов

Рекомендуемая структура проекта (названия можно слегка менять, но смысл должен сохраниться):

```
StudentPortal.Diagnostics/
├─ Program.cs
├─ Middleware/
│  ├─ ErrorHandlingMiddleware.cs
│  ├─ TokenMiddleware.cs
│  └─ TokenExtensions.cs
├─ Services/
│  ├─ IDateTimeService.cs
│  ├─ DateTimeService.cs
│  ├─ IEnvironmentReportService.cs
│  ├─ EnvironmentReportService.cs
│  └─ ServiceCollectionExtensions.cs
└─ Properties/launchSettings.json
```

Допустимый минимальный каркас интерфейсов и сигнатур:

```
public interface IDateTimeService
{
    string GetDate();
    string GetTime();
}
```

```
public class DateTimeService : IDateTimeService
{
    public string GetDate() => DateTime.Now.ToShortDateString();
    public string GetTime() => DateTime.Now.ToShortTimeString();
}
```

```
public class TokenMiddleware
{
    public TokenMiddleware(RequestDelegate next, string pattern) { ... }
    public Task InvokeAsync(HttpContext context) { ... }
}
```

```
public static class TokenExtensions
{
    public static IApplicationBuilder UseToken(this IApplicationBuilder app, string pattern) {
... }
}
```

```
public static class ServiceCollectionExtensions
{
    public static IServiceCollection AddStudentPortalServices(this IServiceCollection services) {
... }
}
```

7. Требования к построению pipeline

- ErrorHandlingMiddleware должен стоять первым, чтобы видеть результат работы следующих компонентов и корректно реагировать на 403 и 404.
- Inline middleware на Use() должен находиться до основных ветвей, чтобы оборачивать их выполнение.
- UseWhen(...) должен только расширять обработку и возвращать запрос в основной pipeline.
- MapWhen(...) должен отделять запросы особого формата в самостоятельную ветку.
- Ветка /secure должна включать токен-проверку раньше терминального обработчика защищенного ресурса.
- Нельзя одновременно бездумно писать ответ через Response.WriteAsync(...) и затем продолжать next, если это ломает логику ответа.

8. Отдельное задание по окружению

Студент должен продемонстрировать, что понимает различие сред Development и Production. Для этого нужно:

11. вывести в /env значение EnvironmentName;
12. добавить в ответ ApplicationName, ContentRootPath и WebRootPath;
13. запустить приложение минимум в одной среде Development и описать, как меняется поведение или вывод;
14. в отчете указать, где именно задается среда запуска (например, через launchSettings.json или переменные окружения).

9. Отдельное задание по DI и IServiceCollection

В рамках этой части студент должен не только зарегистрировать собственные сервисы, но и показать, что понимает саму коллекцию IServiceCollection как реестр зависимостей приложения.

15. Создать минимум два собственных сервиса.
16. Зарегистрировать их до вызова builder.Build().
17. Вынести регистрацию в extension method для IServiceCollection.
18. На маршруте /di/services вывести количество зарегистрированных сервисов и список не менее 10 записей с типом сервиса и lifetime.
19. В одном месте использовать собственный сервис для формирования ответа пользователю.

10. Что должно войти в отчет студента

- скриншот или список успешных запросов к обязательным маршрутам;
- скриншот /unknown и /secure/report без токена;
- фрагмент Program.cs с итоговым порядком middleware;
- краткое объяснение разницы между UseWhen и MapWhen;
- объяснение, почему ErrorHandlingMiddleware должен стоять раньше остальных;
- перечень собственных сервисов и место их регистрации;
- вывод по работе: что было самым важным в понимании request pipeline.

11. Критерии оценивания

Критерий	Баллы	Что проверяет преподаватель
Проект запускается и содержит все обязательные маршруты	15	Работоспособность и корректные ответы
Inline middleware на Use() реализован корректно	10	Есть логика до и после next
UseWhen и MapWhen применены осмысленно и не перепутаны	15	Студент понимает различие между реинтеграцией и отдельной веткой
Map() и вложенные ветки построены корректно	15	Структурированность URL и ветвей
Custom middleware-классы и extension methods оформлены корректно	15	TokenMiddleware, ErrorHandlingMiddleware, UseToken(...)
Работа с IWebHostEnvironment выполнена корректно	10	Есть вывод окружения и связанных путей
Регистрация и использование сервисов через IServiceCollection	10	Есть сервисы, extension method и демонстрация /di/services
Отчет и объяснение порядка pipeline	10	Есть осмысленный анализ, а не только код

12. Дополнительные задания для сильных студентов

- Сделать /di/services в виде HTML-таблицы с тремя колонками: ServiceType, Lifetime, ImplementationType.
- Добавить отдельную защищенную ветку /secure/admin и вложенный маршрут /secure/admin/report.
- Сделать различное содержимое ответа /env для Development и Production.
- Вынести часть inline middleware в отдельный класс и сравнить, где удобнее поддержка.
- Подключить ILogger и логировать шаги обработки запроса в консоль более формально.

Итог для студента

Сдать нужно не просто набор файлов, а понятную демонстрацию логики request pipeline. В отчете студент должен уверенно объяснить разницу между Use, UseWhen, MapWhen и Map, а также порядок включения middleware.